



LARAVEL 6

ELOQUENT ORM

Contents

- Defining Models
- Eloquent Model Conventions
- Retrieving Models
- Inserting & Updating Models
- Deleting Models
- Relationships
- Querying Relations
- Inserting & Updating Related Models

Introduction

- Each database table has a corresponding "Model" which is used to interact with that table.
- Models allow you to query for data in your tables, as well as insert new records into the table.



Defining Models

```
php artisan make:model Flight
```

```
php artisan make:model Flight --migration
```

```
php artisan make:model Flight -m
```

flights table

- Plural name of the class will be used as the table name unless another name is explicitly specified.

AirTrafficController model



air_traffic_controllers table

```
use Illuminate\Database\Eloquent\Model;
class Comment extends Model
{
    //
}
```

comments table

```
use Illuminate\Database\Eloquent\Model;
class Comment extends Model
{
    protected $table = 'my_comments';
}
```

Eloquent Model Conventions (1)

- Eloquent will also assume that each table has a **primary key column named id**
 - You may define a protected **\$primaryKey** property to override this convention.
 - Eloquent assumes that the primary key is an incrementing integer value

```
#[Table(key: 'flight_id')]
class Flight extends Model
{
    // ...
}
```

```
#[WithoutIncrementing]
class Flight extends Model
{
    // ...
}
```

```
#[Table(key: 'uuid', keyType: 'string', incrementing: false)]
class Flight extends Model
{
    // ...
}
```

Eloquent Model Conventions (2)

- By default, Eloquent expects **created_at** and **updated_at** columns to exist on your tables. If you do not wish to have these columns automatically managed by Eloquent, set **timestamps** to **false** on your model's **Table** attribute:

```
#[Table(timestamps: false)]  
class Flight extends Model  
{  
    // ...  
}
```

```
#[WithoutTimestamps]  
class Flight extends Model  
{  
    // ...  
}
```

- If you need to customize the names of the columns used to store the timestamps, you may set the **CREATED_AT** and **UPDATED_AT** constants in your model:

```
class Flight extends Model {  
    const CREATED_AT = 'creation_date';  
    const UPDATED_AT = 'last_update';  
}
```

Eloquent Model Conventions (3)

- All Eloquent models will use the default database connection configured for your application. If you would like to specify a different connection for the model:

```
#[Connection( 'mysql' )]  
class Flight extends Model  
{  
    // ...  
}
```

- If you would like to define the default values for some of your model's attributes, you may define an `$attributes` property on your model:

```
...  
protected $attributes = [  
    'delayed' => false,  
];  
...
```



Retrieving Models (1)

Since Eloquent models are query builders, you should review all of the methods provided by Laravel's [query builder](#). You may use any of these methods when writing your Eloquent queries.

```
$users = User::all();  
  
$articles = Article::where('user_id', 1)  
    ->orderBy('title', 'desc')  
    ->take(4)  
    ->get();
```


Retrieving Models (2)

■ Refreshing Models

- The **fresh** method will re-retrieve the model from the database. The existing model instance will not be affected
- The **refresh** method will re-hydrate the existing model using fresh data from the database (include relationships)

```
$flight = App\Models\Flight::where('number', 'FR 900')->first();  
$flight->number = 'FR 456';  
$flight->refresh();  
$flight->number; // "FR 900"
```

■ Chunking Results

```
Flight::chunk(200, function ($flights) {  
    foreach ($flights as $flight) { // }  
});
```

■ Using Cursors

```
foreach (Flight::where('foo', 'bar')->cursor() as $flight) { // }
```

Retrieving Single Models / Aggregates

- Retrieve single records using **find**, **first**, or **firstWhere**, these methods return a single model instance:

```
// Retrieve a model by its primary key...
```

```
$flight = App\Models\Flight::find(1);
```

```
// Retrieve the first model matching the query constraints...
```

```
$flight = App\Models\Flight::where('active', 1)->first();
```

```
// Shorthand for retrieving the first model matching the query constraints...
```

```
$flight = App\Models\Flight::firstWhere('active', 1);
```

- You may also call the **find** method with an array of primary keys, which will return a collection of the matching records:

```
$flights = App\Models\Flight::find([1, 2, 3]);
```

```
$count = App\Models\Flight::where('active', 1)->count();
```

```
$max = App\Models\Flight::where('active', 1)->max('price');
```



Inserting & Updating Models

Insert

```
$flight = new Flight;  
  
$flight->name = $request->name;  
  
$flight->save();
```

Update

```
$flight = App\Models\Flight::find(1);  
  
$flight->name = 'New Flight Name';  
  
$flight->save();
```

Mass Update

```
App\Models\Flight::where('active', 1)  
    ->where('destination', 'San Diego')  
    ->update(['delayed' => 1]);
```

Examining Attribute Changes (1)

- Eloquent provides the `isDirty`, `isClean`, and `wasChanged` methods to examine the internal state of your model and determine how its attributes have changed from when they were originally loaded.
 - The `isDirty` method determines if **any attributes have been changed since the model was loaded**
 - The `isClean` method is the opposite of `isDirty`

```
$user = User::create([  
    'first_name' => 'Taylor',  
    'last_name' => 'Otwell',  
    'title' => 'Developer',  
]);  
$user->title = 'Painter';  
$user->isDirty(); // true  
$user->isDirty('title'); // true
```

```
$user->isDirty('first_name'); // false  
$user->isClean(); // false  
$user->isClean('title'); // false  
$user->isClean('first_name'); // true  
$user->save();  
  
$user->isDirty(); // false  
$user->isClean(); // true
```



Examining Attribute Changes (2)

- The **wasChanged** method determines if any attributes were changed when the model was last saved within the current request cycle.
- The **getOriginal** method returns an array containing the original attributes of the model regardless of any changes since the model was loaded

```
$user = User::create([
    'first_name' => 'Taylor',
    'last_name' => 'Otwell',
    'title' => 'Developer',
]);
$user->title = 'Painter';
$user->getOriginal('title'); // Developer
$user->getOriginal(); // Array of original attributes...

$user->save();
$user->wasChanged(); // true
$user->wasChanged('title'); // true
$user->wasChanged('first_name'); // false
```

Other Creation Methods

■ firstOrCreate/ firstOrNew

```
// Retrieve flight by name, or create it if it doesn't exist...
$flight = App\Models\Flight::firstOrCreate(['name' => 'Flight 10']);

// Retrieve flight by name, or create it with the name, delayed, and arrival_time attributes...
$flight = App\Models\Flight::firstOrCreate(
    ['name' => 'Flight 10'],
    ['delayed' => 1, 'arrival_time' => '11:30'] );

// Retrieve by name, or instantiate...
$flight = App\Models\Flight::firstOrNew(['name' => 'Flight 10']);

// Retrieve by name, or instantiate with the name, delayed, and arrival_time attributes...
$flight = App\Models\Flight::firstOrNew(
    ['name' => 'Flight 10'],
    ['delayed' => 1, 'arrival_time' => '11:30'] );
```

Other Creation Methods (2)

■ updateOrCreate

```
// If there's a flight from Oakland to San Diego, set the price to $99.  
// If no matching model exists, create one.  
$flight = App\Models\Flight::updateOrCreate(  
    ['departure' => 'Oakland', 'destination' => 'San Diego'],  
    ['price' => 99, 'discounted' => 1]  
);
```



Deleting Models

```
$flight = App\Models\Flight::find(1);  
$flight->delete();
```

```
App\Models\Flight::destroy(1);  
App\Models\Flight::destroy(1, 2, 3);  
App\Models\Flight::destroy([1, 2, 3]);  
App\Models\Flight::destroy(collect([1, 2, 3]));
```

- The **destroy** method loads each model individually and calls the **delete** method on them so that the **deleting** and **deleted** events are fired.

```
$deletedRows = App\Models\Flight::where('active', 0)->delete();
```

- When executing a mass **delete statement** via Eloquent, the **deleting** and **deleted model events** will not be fired for the deleted models. This is because the models are never actually retrieved when executing the delete statement.

Relationships

- One To One
- One To Many
- One To Many (Inverse)
- Many To Many



One To One

Illuminate\Database\Eloquent\Model

Illuminate\Database\Eloquent\Model\Relations\HasOne

```
class User extends Model {  
    /** * Get the phone record associated with the user. */  
    public function phone() : HasOne{  
        return $this->hasOne(Phone::class);  
    }  
}  
  
$phone = User::find(1)->phone;
```

- The Phone model is automatically assumed to have a `user_id` foreign key

```
return $this->hasOne(Phone::class, 'foreign_key');
```

```
return $this->hasOne(Phone::class, 'foreign_key', 'local_key');
```

One To One

- Defining The Inverse Of The Relationship

```
class Phone extends Model {  
    /** * Get the user that owns the phone. */  
    public function user():BelongsTo {  
        return $this->belongsTo(User::class);  
    }  
}
```

```
$user = Phone::find(1)->user;
```

```
return $this->belongsTo(User::class, 'foreign_key');
```

```
return $this->belongsTo(User::class, 'foreign_key', 'owner_key');
```

One To Many

- Eloquent will take the "snake case" name of the owning model and **suffix it with _id**. So, for this example, Eloquent will assume the foreign key on the Comment model is **post_id**

```
class Post extends Model {  
    /** * Get the comments for the blog post. */  
    public function comments(): HasMany {  
        return $this->hasMany(Comment::class);  
    }  
}
```

```
return $this->hasMany(Comment::class, 'foreign_key');  
return $this->hasMany(Comment::class, 'foreign_key', 'local_key');
```

```
$comments = Post::find(1)->comments;  
foreach ($comments as $comment) { // }
```

```
$comment = Post::find(1)->comments()->where('title', 'foo')->first();
```



One To Many (Inverse)

```
class Comment extends Model {  
    /** * Get the post that owns the comment. */  
    public function post(): BelongsTo {  
        return $this->belongsTo(Post::class);  
    }  
}
```

```
return $this->belongsTo(Post::class, 'foreign_key');  
return $this->belongsTo(Post::class, 'foreign_key', 'owner_key');
```

```
$comment = App\Models\Comment::find(1);  
echo $comment->post->title;
```

Has One of Many

- Retrieve the "latest" or "oldest" related model of the relationship

```
public function latestOrder(): HasOne
{
    return $this->hasOne(Order::class)->latestOfMany();
}
```

```
public function oldestOrder(): HasOne
{
    return $this->hasOne(Order::class)->oldestOfMany();
}
```

- By default, the *latestOfMany* and *oldestOfMany* methods will retrieve the latest or oldest related model based on the model's primary key, which must be sortable.

```
public function largestOrder(): HasOne
{
    return $this->hasOne(Order::class)->ofMany('price', 'max');
}
```

Column Function
↓ ↓
 'price', 'max';



Converting "Many" Relationships to Has One Relationships

```
/**
 * Get the user's orders.
 */
public function orders(): HasMany
{
    return $this->hasMany(Order::class);
}

/**
 * Get the user's largest order.
 */
public function largestOrder(): HasOne
{
    return $this->orders()->one()->ofMany('price', 'max');
}
```

```
public function latestDeployment(): HasOneThrough
{
    return $this->deployments()->one()->latestOfMany();
}
```



Has One Through (1)

```
class Mechanic extends Model
{
    /**
     * Get the car's owner.
     */
    public function carOwner(): HasOneThrough
    {
        return $this->hasOneThrough(Owner::class,
                                    Car::class);
    }
}
```

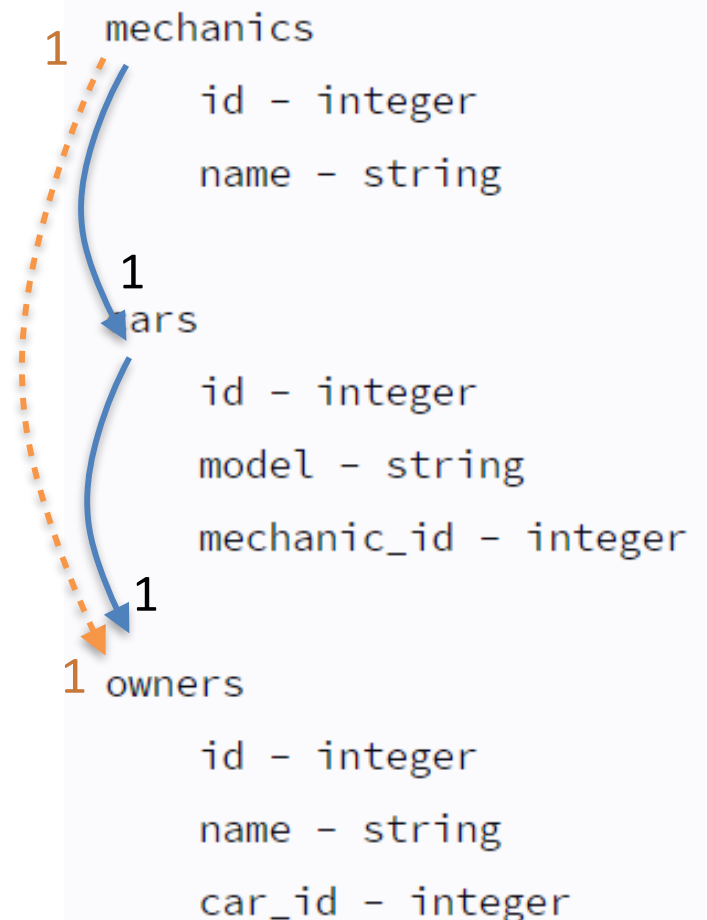
intermediate model

final model

if the relevant relationships have already been defined on all of the models involved in the relationship,

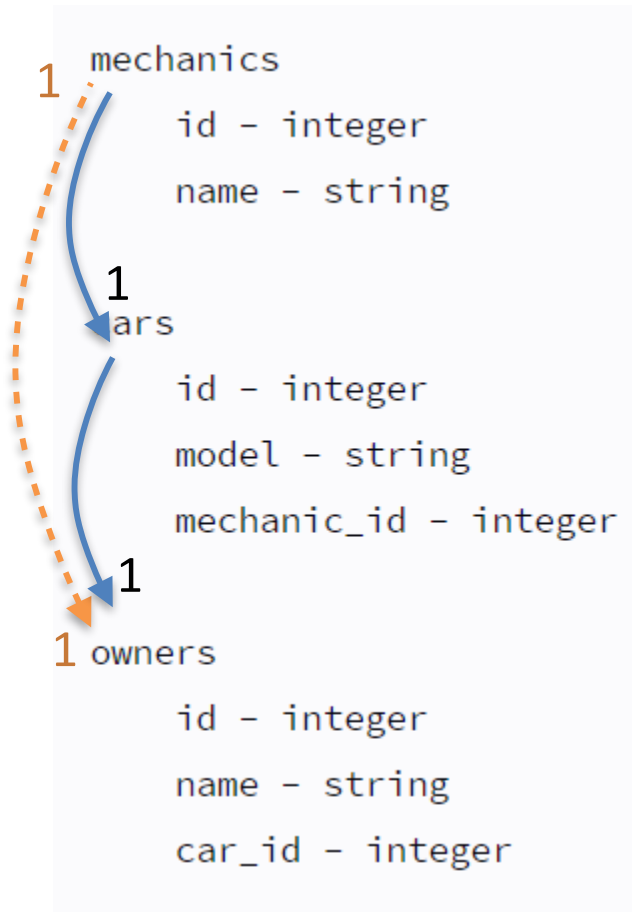
```
// String based syntax...
return $this->through('cars')->has('owner');

// Dynamic syntax...
return $this->throughCars()->hasOwner();
```





Has One Through (2)



```
class Mechanic extends Model
{
    /**
     * Get the car's owner.
     */
    public function carOwner(): HasOneThrough
    {
        return $this->hasOneThrough(
            Owner::class,
            Car::class,
            'mechanic_id', // Foreign key on the cars table...
            'car_id', // Foreign key on the owners table...
            'id', // Local key on the mechanics table...
            'id' // Local key on the cars table...
        );
    }
}
```

Has Many Through (1)

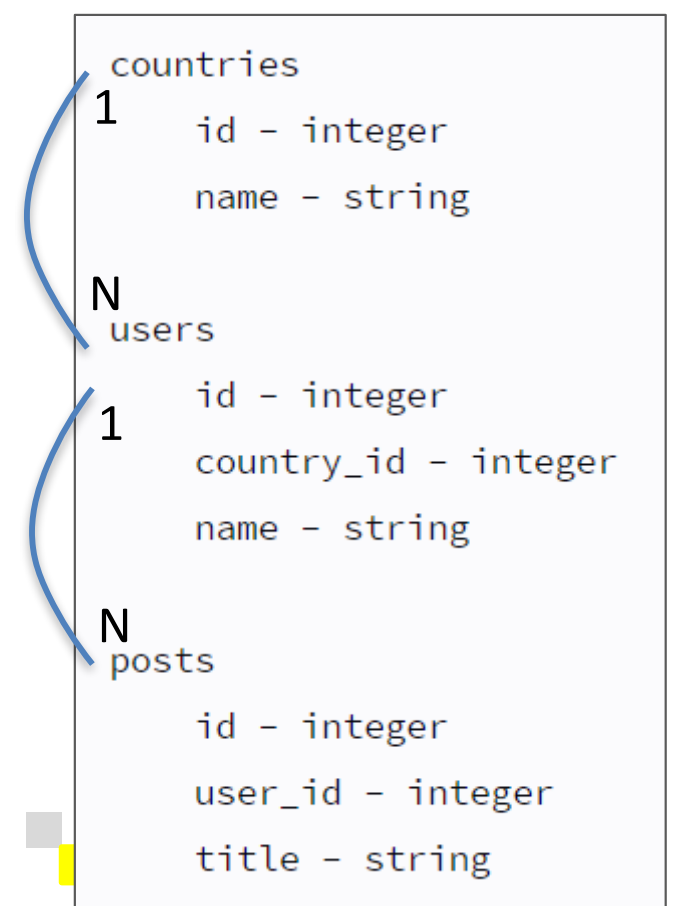
```
class Country extends Model
{
  public function post(): HasManyThrough
  {
    return $this->hasManyThrough(Post::class, User::class);
  }
}
```

// String based syntax...

```
return $this->through('users')->has('posts');
```

// Dynamic syntax...

```
return $this->throughUsers()->hasPosts();
```



```
countries
1
  id - integer
  name - string
N
users
1
  id - integer
  country_id - integer
  name - string
N
posts
  id - integer
  user_id - integer
  title - string
```



Has Many Through (2)

```
countries
  id - integer
  name - string

users
  id - integer
  country_id - integer
  name - string

posts
  id - integer
  user_id - integer
  title - string
```

```
class Country extends Model
{
  public function post(): HasManyThrough
  {
    return $this->hasManyThrough(
      Post::class, User::class,
      'country_id', // Foreign key on the users table...
      'user_id', // Foreign key on the posts table...
      'id', // Local key on the countries table...
      'id' // Local key on the users table...);
  }
}
```



Many To Many

```
users
  id - integer
  name - string
```

```
roles
  id - integer
  name - string
```

```
role_user
  user_id - integer
  role_id - integer
```

```
class User extends Model
{
    /**
     * The roles that belong to the user.
     */
    public function roles(): BelongsToMany
    {
        return $this->belongsToMany(Role::class);
    }
}
```

```
class Role extends Model
{
    public function users(): BelongsToMany
    {
        return $this->belongsToMany(User::class);
    }
}
```

```
$user = User::find(1);
foreach ($user->roles as $role) { // }
$roles = User::find(1)->roles()->orderBy('name')->get();
```

Retrieving Intermediate Table Columns

- The **pivot** attribute contains a model representing the intermediate table, and may be used like any other Eloquent model.

```
$user = App\Models\User::find(1);  
foreach ($user->roles as $role) {  
    echo $role->pivot->created_at;  
}
```

The column of role_user table

```
return $this->belongsToMany(Role::class)  
    ->withPivot('column1', 'column2');
```

```
return $this->belongsToMany(Podcast::class)  
    ->as('subscription')  
    ->withTimestamps();
```

rename pivot table to subscription

```
$users = User::with('podcasts')->get();  
foreach ($users->flatMap->podcasts as $podcast) {  
    echo $podcast->subscription->created_at;  
}
```

Filtering Relationships Via Intermediate Table Columns

- You can also filter the results returned by `belongsToMany` using the `wherePivot`, `wherePivotIn`, and `wherePivotNotIn` methods when defining the relationship:

```
return $this->belongsToMany(Role::class)->wherePivot('approved', 1);  
  
return $this->belongsToMany(Role::class)->wherePivotIn('priority', [1, 2]);  
  
return $this->belongsToMany(Role::class)->wherePivotNotIn('priority', [1, 2]);
```

Polymorphic Relationships

- <https://laravel.com/docs/13.x/eloquent-relationships#polymorphic-relationships>

Querying Relations

- Since all types of Eloquent relationships are defined via methods, you may call those methods to obtain an instance of the relationship without actually executing the relationship queries
- In addition, all types of Eloquent relationships also **serve as query builders, allowing you to continue to chain constraints onto the relationship query before finally executing the SQL against your database**

Querying Relations

- You may query the posts relationship and add additional constraints to the relationship like so:

```
$user = App\Models\User::find(1);  
$user->posts()->where('active', 1)->get();
```

- You are able to use any of the query builder methods on the relationship

Relationship Methods Vs. Dynamic Properties

- If you do not need to add additional constraints to an Eloquent relationship query, you may access the relationship as if it were a property

```
$user = App\Models\User::find(1);  
foreach ($user->posts as $post) { // }
```

- Dynamic properties are "lazy loading", meaning they will only load their relationship data when you actually access them. Because of this, developers often use eager loading to pre-load relationships they know will be accessed after loading the model.

Querying Relationship Existence

```
// Retrieve all posts that have at least one comment...
$posts = App\Models\Post::has('comments')->get();
// Retrieve all posts that have three or more comments...
$posts = App\Models\Post::has('comments', '>=', 3)->get();
$result = App\Models\Post::has('tags')->orHas('comments')->get()->toArray();
```

the name of the relationship

- Nested has statements may also be constructed using "dot" notation. For example, you may retrieve all posts that have at least one comment and vote:

```
// Retrieve posts that have at least one comment with votes...
$posts = App\Models\Post::has('comments.votes')->get();
```

the **comments** relationship of Post model

the **votes** relationship of Comment model

Querying Relationship Existence

- You may use the **whereHas** and **orWhereHas** methods to put "**where**" conditions on your has queries. These methods allow you to add customized constraints to a relationship constraint, such as checking the content of a comment:

```
use Illuminate\Database\Eloquent\Builder;
// Retrieve posts with at least one comment containing words like foo%...
$posts = App\Models\Post::whereHas('comments', function (Builder $query)
{ $query->where('content', 'like', 'foo%'); })->get();

// Retrieve posts with at least ten comments containing words like foo%...
$posts = App\Models\Post::whereHas('comments',
    function (Builder $query) {
        $query->where('content', 'like', 'foo%');
    }, '>=', 10)->get();
```

Querying Relationship Absence

- When accessing the records for a model, you may wish to limit your results based on the absence of a relationship. To do so, you may pass the name of the relationship to the `doesn'tHave` and `orDoesntHave` methods

```
$posts = App\Models\Post::doesn'tHave('comments')->get();
```

- you may use the `whereDoesntHave` and `orWhereDoesntHave` methods to put "where" conditions on your `doesn'tHave` queries

```
use Illuminate\Database\Eloquent\Builder;

$posts = App\Models\Post::whereDoesntHave('comments',
    function (Builder $query) {
        $query->where('content', 'like', 'foo%');
    })->get();
```



Aggregating Related Models

■ Counting Related Models

- If you want to count the number of results from a relationship without actually loading them you may use the **withCount** method, which will place a **{relation}_count** column on your resulting models. For example:

```
$posts = App\Models\Post::withCount('comments')->get();  
foreach ($posts as $post) {  
    echo $post->comments_count;  
}
```

```
use Illuminate\Database\Eloquent\Builder;  
$posts = App\Models\Post::withCount(['votes',  
    'comments' => function (Builder $query) {  
        $query->where('content', 'like', 'foo%');  
    }])->get();  
  
echo $posts[0]->votes_count;  
echo $posts[0]->comments_count;
```

Aggregating Related Models

■ Counting Related Models

- Using the **loadCount** method, you may load a relationship count after the parent model has already been retrieved:

```
$book = App\Models\Book::first();  
  
$book->loadCount('genres');
```

■ Other Aggregate Functions

- Eloquent provides **withMin**, **withMax**, **withAvg**, and **withSum**. These methods will place a **{relation}_{function}_{column}** column on your resulting models.

```
$posts = App\Models\Post::withSum('comments', 'votes')->get();  
foreach ($posts as $post) {  
    echo $post->comments_sum_votes;  
}
```



Eager Loading

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Book extends Model {
    /** * Get the author that wrote the book. */
    public function author() {
        return $this->belongsTo(Author::class);
    }
}
```

```
$books = Book::all();
foreach ($books as $book) {
    echo $book->author->name;
}
```

Lazy Loading

if we have 25 books, the
code above would run
1+25 queries

```
$books = Book::with('author')->get();
foreach ($books as $book) {
    echo $book->author->name;
}
```

Eager Loading

select * from books **2 queries**

select * from authors where id in (1, 2, 3, 4, 5, ...)

```
(['author', 'publisher'] )->get();
```


Inserting & Updating Related Models

```
$comment = new Comment(['message' => 'A new comment.']);  
$post = Post::find(1);  
$post->comments()->save($comment);
```

```
$post->comments()->saveMany([  
    new Comment(['message' => 'A new comment.']),  
    new Comment(['message' => 'Another comment.']),  
]);
```

```
$post->comments()->save($comment);  
$post->refresh();
```

```
// All comments, including the newly saved comment...  
$post->comments;
```

Inserting & Updating Related Models

- If you would like to save your model and all of its associated relationships, you may use the push method

```
$post = Post::find(1);  
$post->comments[0]->message = 'Message';  
$post->comments[0]->author->name = 'Author Name';  
$post->push();
```

- In addition to the **save** and **saveMany** methods, you may also use the **create** method

```
$post = Post::find(1);  
$comment = $post->comments()->create([ 'message' => 'A new comment.', ]);  
$post->comments()->createMany([  
    [ 'message' => 'A new comment.', ],  
    [ 'message' => 'Another new comment.', ],  
]);
```

Q&A